

Spesifikasi Seleksi 3 Asisten Laboratorium Programming 2025

Tim Asisten Lab Pemrograman 2022

Tgl. Revisi Terakhir : -

Tgl. Mulai : **30 Juli 2025** pukul **22.10 WIB**

Tenggat Waktu : **22 Agustus 2025** pukul **22.10 WIB**

Revisi:

- 15 Agustus 2025 : Penambahan tautan [pengumpulan](#).

Daftar Isi

Daftar Isi	1
Deskripsi Persoalan	3
Spesifikasi Utama	4
F01 - Monolith (FE)	5
1. Register Page	5
2. Login Page	5
3. Browse Course Page	5
4. Course Detail Page	5
5. My Course Page	5
6. Course Module Page	6
F02 - Monolith (BE)	6
1. Register	6
2. Login	6
3. Browse Course	7
4. Buy Course	7
5. My Course	7
6. Module Management	7
F03 - REST API (BE)	7
1. CRUD Course	8
2. CRUD Module	8
3. Auth	8
4. CRUD User	9
Kontrak API	9
1. CRUD Course	9
2. CRUD Module	13
3. Auth	18
4. CRUD User	20
Error Handling Standards	23
1. Error Codes	23
2. Error Messages	23
Bonus	23
B01 - OWASP	24
B02 - Deployment	24
B03 - Polling	24

B04 - Caching	24
B05 - Lighthouse	25
B06 - Responsive Layout	25
B07 - Dokumentasi API	25
B08 - SOLID	25
B09 - Automated Testing	25
B10 - Fitur Tambahan	26
B11 - Ember	27
Batasan dan Deliverables	28
Batasan	28
Deliverables	28

Deskripsi Persoalan



Grocademy - Platform Belajar milik Gro

Di suatu hari yang cerah, Gro sedang asyik menikmati masa tuanya dengan santai di rumah. Seperti kebanyakan orang di era digital, ia menghabiskan waktu dengan berselancar di internet sambil menyeruput teh hangat dan sesekali mengomel pada layar komputer yang lambat. Tiba-tiba, mata Gro terbelalak ketika melihat sebuah berita viral yang sedang trending di mana-mana: "Chat-ROIFA, AI Terbaru Dr. Neroifa Meledak! 100 Juta Pengguna dalam 1 Hari!" "Astaga!" teriak Gro sambil hampir tersedak tehnya. "Neroifa benar-benar gila! Dulu dia cuma bikin mesin cuci yang bisa terbang, sekarang malah bikin AI yang bisa ngobrol kayak manusia!"

Gro langsung penasaran dan mulai mencoba Chat-ROIFA. Ia terpukau melihat betapa cerdasnya AI ciptaan sahabat lamanya itu. Chat-ROIFA bisa menjawab segala pertanyaan, dari resep banana smoothie hingga cara menghitung pajak (yang selalu bikin pusing kepala seperti di negara Wakanda). Saat sedang kagum dengan kehebatan Chat-ROIFA, pandangan Gro tertuju pada para Nimons yang sedang berlarian di halaman belakang rumah. Kebin sedang mengejar kupu-kupu sambil berteriak "BANANA!", Stewart sibuk main gitar dengan suara fals yang mengerikan, Pop sedang memeluk boneka unicorn sambil tertawa tanpa sebab, dan Toto... well, Toto sedang terjebak di dalam ember cat (lagi).

"Hmm..." gumam Gro sambil menggaruk dagunya. "Selama ini aku selalu bingung cara investasi yang cocok buat para Nimons. Bitcoin? Mereka kira itu nama pisang baru. Saham? Mereka pikir itu semacam kue. Emas? Langsung dimakan." Gro terus merenungkan sambil memperhatikan tingkah laku Nimons yang... unik. "Tapi tunggu dulu," pikirnya. "Kalau investasi uang memang

nggak cocok buat mereka, bagaimana kalau investasi otak? Nimons itu sebenarnya cerdas, cuma perlu diasah aja!"

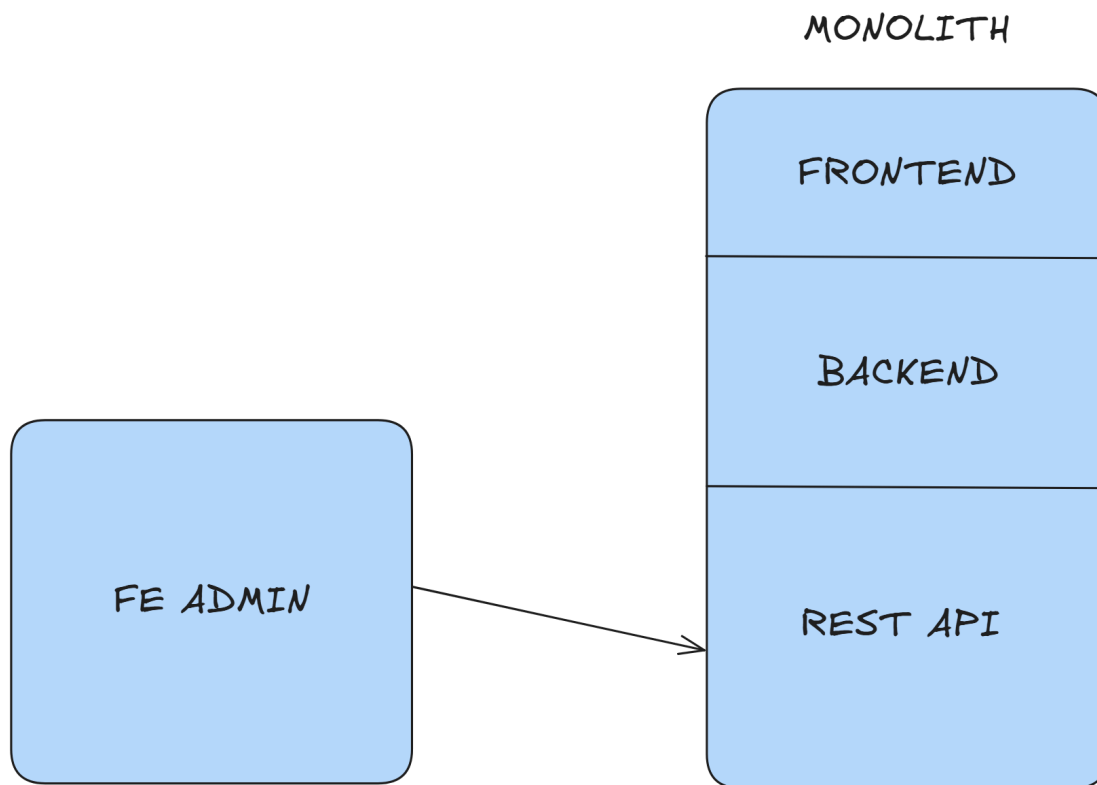
Mata Gro berbinar-binar. "Itu dia! Daripada mereka buang-buang waktu nyari pisang atau nyanyi lagu aneh, mending mereka belajar hal-hal berguna! Siapa tahu suatu hari nanti mereka bisa jadi mengalahkan Chat-ROIFA!" Kebetulan saat itu Luiy, istri Gro yang bijaksana, lewat sambil membawa segelas jus jeruk. "Sayang, kamu lagi mikirin apa sampai senyum-senyum sendiri gitu?" tanyanya. "Luiy!" seru Gro dengan antusias. "Aku punya ide gila! Bagaimana kalau kita bikin platform belajar khusus buat para Nimons? Namanya... Grocademy! Tempat di mana mereka bisa belajar segala macam hal dengan cara yang menyenangkan!"

Luiy tersenyum. "Ide bagus! Tapi kamu yakin bisa bikin sendiri? Terakhir kali kamu coba bikin website, hasilnya malah jadi kalkulator yang cuma bisa hitung 1+1." Gro terdiam sejenak, lalu tertawa. "Hahaha, kamu benar! Makanya aku butuh bantuan. Aku akan mencari developer-developer muda berbakat yang bisa membantu mewujudkan mimpi ini!" Gro lalu berdiri dengan penuh semangat, menghadap ke arah para Nimons yang masih sibuk dengan aktivitas random mereka. "Dengar ya, Nimons! Bapak akan bikin sesuatu yang keren buat kalian. Sebuah tempat di mana kalian bisa belajar tanpa harus duduk diam di kelas yang membosankan. Kalian bisa belajar sambil tetap jadi diri kalian yang... err... unik!"

Kebin langsung berlari menghampiri sambil berteriak, "BAPAAAA! Grocademy punya pisang nggak?" "Ehh... pasti ada course tentang pisang, Kebin!" jawab Gro sambil mengelus kepala Kebin. Stewart ikut mendekat sambil membawa gitarnya, "Course musik juga ada kan, Bapak? Biar aku bisa main gitar lebih keren!" "Tentu saja! Bahkan mungkin ada course khusus cara bermain gitar tanpa bikin tetangga pindah rumah," canda Gro. Pop yang masih memeluk unicornnya bertanya dengan suara imut, "Bapak, nanti di Grocademy ada teman-teman unicorn nggak?" "Pasti ada course tentang mitologi dan makhluk fantasi, Pop!"

Sementara Toto yang baru saja berhasil keluar dari ember cat berteriak, "GELATO! GELATO!" "Iya Toto, mungkin nanti ada course masak gelato juga," jawab Gro sambil geleng-geleng kepala. Dan begitulah, dari sebuah momen sederhana saat melihat kehebatan Chat-ROIFA, Gro mendapat inspirasi untuk menciptakan Grocademy - sebuah platform pembelajaran digital yang tidak hanya edukatif, tetapi juga menyenangkan dan cocok untuk semua kalangan, terutama para Nimons dengan segala keunikan mereka. Tapi tentu saja, Gro tidak bisa mewujudkan mimpi ini sendirian. Ia membutuhkan bantuan para developer muda berbakat yang memiliki visi sama: menciptakan tempat belajar yang tidak membosankan, mudah diakses, dan bisa membuat siapa saja (bahkan Nimons sekalipun) semangat menambah ilmu.

Spesifikasi Utama



Ilustrasi Arsitektur Program

Pada persoalan kali ini, kalian diminta untuk membuat sebuah **monorepo** yang berisi aplikasi website **monolith**. Monolith adalah sebuah model arsitektur di mana sebuah sistem dibangun dalam satu codebase yang sama. Tentunya, terdapat bagian **Frontend** yang berfungsi untuk menyediakan view website dan bagian **Backend** yang berfungsi untuk menyediakan business logic dan routing.

Tidak hanya menyediakan business logic dan routing terhadap Frontend, Backend juga harus menyediakan **REST API** yang dapat digunakan oleh website lain, yaitu **Frontend Admin**. Dengan demikian, Frontend Admin dapat langsung berinteraksi dengan website monolith tanpa perlu membuat service tambahan. Semua source code yang dibuat wajib berada dalam repository yang sama.

Frontend Admin akan disediakan oleh Labpro sehingga kalian **tidak perlu mengimplementasikan Frontend Admin**. Kalian hanya perlu membuat REST API yang dapat menerima *request* dari FE Admin.

Berikut dilampirkan pranala [halaman admin](#) yang sudah disediakan oleh Labpro.

Berikut dilampirkan pranala [github FE admin](#) yang sudah disediakan oleh Labpro. Backend demo akan direset setiap **30 menit** sehingga kalian bebas melakukan manipulasi pada situs tersebut.

F01 - Monolith (FE)

Seluruh spesifikasi di bawah ini merupakan batas **MINIMAL** yang perlu kalian implementasikan. Silahkan gunakan kreativitas kalian untuk menambah maupun mengembangkan fitur-fitur dan halaman yang ada disini. Segala bentuk inovasi akan diapresiasi.

1. Register Page

Sebuah halaman register untuk mendaftarkan pengguna baru dengan input data minimal yaitu:

- Email
- Username
- Nama lengkap (nama depan dan nama belakang)
- Password

2. Login Page

Sebuah halaman *login* yang meminta email/username serta *password*. Setelah pengguna berhasil melakukan *login*, penampilan data *current user* yang bersifat penting, seperti *username* dan *current balance* dibebaskan, yang penting ditampilkan untuk memudahkan user.

3. Browse Course Page

Halaman ini berisi list daftar *course* yang tersedia (baik yang sudah dibeli maupun belum).. Terdapat *search box* yang dapat mencari *course* tertentu berdasarkan kata yang dicari. Kata dapat memuat atribut dari suatu *course* seperti nama *course* atau *instructor*. Agar data yang ditampilkan pada 1 halaman tidak terlalu banyak, maka tambahkan ***pagination*** untuk membatasi jumlah *course* yang ditampilkan pada 1 halaman. *Pagination* wajib dilakukan dari sisi *backend*. *Pagination* yang ada juga wajib untuk memberikan opsi mengenai berapa banyak hasil yang ingin ditampilkan di tiap halamannya (opsi dan sarana untuk melakukan ini dibebaskan).

4. Course Detail Page

Jika pengguna menekan sebuah *course* pada *Browse Course Page*, maka arahkan ke *Course Detail Page* yang berisi:

- Detail lengkap *course* (*title, description, instructor, topics, price*)

- Tombol ke **Module Page** (jika sudah dibeli)
- Tombol **Beli** (jika belum dibeli)
- Tombol **Unduh Sertifikat** (jika sudah menyelesaikan seluruh modul yang ada di sebuah course)

Pembelian langsung dapat dilakukan jika uang pengguna cukup. Uang pengguna hanya dapat ditambahkan melalui FE Admin.

5. My Course Page

Berisi list *course* yang sudah dibeli oleh user. Jika pengguna menekan sebuah *course*, maka arahkan ke *Course Detail Page*. (Fungsionalitas sama dengan *Browse Course Page* tetapi hanya menampilkan course yang sudah dibeli)

6. Course Module Page

Berisi list module dari course yang dipilih sebelumnya. Beberapa Fitur pada halaman ini:

1. User dapat melihat daftar semua module dalam *course* (diurutkan berdasarkan *order* menaik).
2. User dapat membaca/mempelajari isi dari setiap module (PDF viewer / Video Player) atau keduanya.
3. User dapat menandai module sebagai "selesai"
4. *Progress bar* yang menunjukkan berapa persen course yang sudah diselesaikan
5. Setelah semua module selesai, user mendapatkan **sertifikat** yang dapat didownload
6. Untuk format dari sertifikat sendiri dibebaskan, tetapi harus memuat info dinamis berupa
 - a. Username dari pengguna yang menyelesaikan *course* tersebut.
 - b. Judul Course yang diikuti, serta instruktornya.
 - c. Tanggal pengguna pertama kali menyelesaikan seluruh module yang ada.

F02 - Monolith (BE)

1. Register

- Lakukan validasi data yang diterima dari Frontend sebelum masuk ke dalam database
- Email dan username yang diterima harus **unik**
- Apabila email atau username tidak unik, maka kembalikan error ke *Frontend*
- Password yang disimpan di database wajib di-**hash**, dengan metode hashing dibebaskan.
- Gunakan syarat-syarat terkait email, username, dan password untuk menambah keamanan dan kewajiban data. Contoh:
 - Password minimal terdiri dari 8 karakter dengan mengandung huruf dan juga angka.

- Email harus dalam format yang tepat

2. Login

- Lakukan validasi data yang diterima dari *Frontend*
- Login dapat dilakukan menggunakan email atau username yang dikombinasikan dengan password
- Apabila seluruh credentials benar, maka Backend akan mengirimkan **JWT Token** yang akan disimpan di Frontend
- Setiap kali sebuah request API dilakukan dari Frontend, gunakan metode **authorization bearer token**
- Tidak perlu mengirimkan *refresh token*
- Hint: Dapat memanipulasi nilai HttpOnly Cookie atau melakukan penyimpanan token dalam sebuah storage

3. Browse Course

- Daftar course yang dikirimkan merupakan seluruh course yang tersedia
- Kirimkan status dari course tersebut juga (sudah dibeli/belum dibeli)
- Untuk search functionality, gunakan **LIKE search** dengan **case-insensitive**
 - Contoh: course "CDIA Investment" muncul saat diketikkan kata "dia"
- Implementasikan pagination untuk membatasi jumlah data per halaman

4. Buy Course

- Ketika pengguna membeli course maka kurangi uang pengguna pada Database sesuai dengan harga course yang dibeli;
- Perlu diperhatikan bahwa pengguna **tidak dapat** membeli course jika harga course melebihi uang milik pengguna
- Setelah pembelian berhasil, user dapat mengakses semua module dalam course tersebut

5. My Course

- Course-course yang dibeli oleh *current user*
- Pastikan bahwa data yang dikirimkan merupakan pembelian dari orang dengan id tervalidasi, bukan data pembelian semua orang

6. Module Management

- Get all modules dari course tertentu
- *Track progress module* (sudah ditandai selesai/belum)
- Tandai modul sebagai selesai
- Generate sertifikat ketika semua module ditandai selesai

F03 - Monolith (REST API)

Spesifikasi yang ada setelah ini merupakan batas **MINIMAL** yang perlu kalian implementasikan. Pada prinsipnya, kalian boleh menambah ataupun memodifikasi beberapa endpoint yang ada, selama **SELURUH** proses manajemen dari Course serta module dapat berjalan dengan baik dari FE Admin.

1. CRUD Course

Buatlah endpoint yang dapat digunakan untuk menambahkan *course*, mengedit *course*, membaca *course*, dan menghapus *course*. **Endpoint CREATE, UPDATE, DELETE hanya dapat diakses oleh admin.**

Berikut merupakan Informasi course yang disimpan:

- title (required)
- description (required)
- instructor (required)
- topics[] (required)
- price (required)
- thumbnail_image (nullable, URL yang disimpan pada object storage)
- created_at
- updated_at

2. CRUD Module

Buatlah endpoint yang dapat digunakan untuk menambah module, mengedit module, membaca module, dan menghapus module. Module bersifat **many-to-one** dengan course. **Endpoint CREATE, UPDATE, DELETE hanya dapat diakses oleh admin. User non-admin hanya bisa READ modul dari course yang telah dibeli oleh user tersebut.**

Berikut merupakan informasi module yang disimpan:

- course_id (foreign key)
- title (required)
- description (required)
- pdf_content (nullable, URL yang disimpan pada object storage)
- video_content (nullable, URL yang disimpan pada object storage)
- order (required, urutan module dalam course, harus unik di setiap course)
- created_at
- Updated_at

3. Auth

Buatlah endpoint yang dapat digunakan untuk melakukan login dan mendapatkan informasi dari akun tersebut. Khusus untuk akun admin ditulis secara **hardcode** di basis data. Apabila seluruh credentials benar, maka BE akan mengirimkan token JWT yang akan disimpan di client. Setiap kali sebuah request API dilakukan dari client, gunakan metode authorization bearer token. Tidak perlu mengirimkan refresh token. Untuk memudahkan pengecekan, pastikan bahwa masa berlaku token yang ada maksimal di 1 jam saja ketika proses demonstrasi. Apabila lebih dari itu, maka request gagal dan pengguna diminta untuk *login* kembali ke aplikasi. Pastikan bahwa yang dapat masuk ke FE Admin hanya merupakan akun admin. Mekanisme pembatasan dibebaskan.

4. CRUD User

Buatlah endpoint yang dapat digunakan untuk membaca user, mengedit user (termasuk balance), serta menghapus user. **Endpoint CREATE, READ, UPDATE, DELETE hanya dapat diakses oleh admin.** Pastikan bahwa admin tidak bisa menghapus dirinya sendiri.

Note: Perhatikan *best practice* dalam penyimpanan media seperti video dan cover image. **Jangan menyimpan blob atau binary file langsung di dalam basis data.** *Object storage* dapat berupa layanan eksternal (AWS S3, *Google Cloud*, dll.) atau sistem penyimpanan file lokal. Kalian juga perlu memastikan jika sebuah course dihapus atau mengganti media, maka media yang lama harus dihapus.

Kontrak API

Berikut dilampirkan kontrak API yang perlu dipenuhi. Beberapa kontrak disini dibuat karena memang digunakan di Web Admin, tetapi ada pula Endpoint yang bersifat “saran” agar kalian memiliki panduan mengenai hal-hal yang mungkin perlu dibuat. Silahkan eksplorasi lebih lanjut untuk memastikan bahwa endpoint yang kalian buat dapat digunakan dengan baik pada [frontend](#) yang telah disediakan. Apabila terdapat masalah atau bug pada frontend yang disediakan, kalian boleh melakukan *Pull Request* ke [repository frontend](#).

Untuk diingat sekali lagi bahwa pada prinsipnya segala bentuk modifikasi **DIPERBOLEHKAN**, selama website kalian masih dapat bekerja dengan baik ketika diakses menggunakan FE Admin. (Misal menambahkan informasi tertentu di response body untuk memperoleh informasi tertentu)

Asumsikan bahwa Admin juga bisa login di website utama kalian dan dianggap sebagai pengguna pada umumnya.

1. CRUD Course

POST /api/courses	
Request	
Header	Content-Type: multipart/form-data
	Authorization: Bearer <token>
Body	<pre>title: string description: string instructor: string topics: string[] price: number thumbnail_image: <binary image file> null</pre>
Response	
<pre>{ "status": "success" "error", "message": "string", "data": { "id": "string", "title": "string", "description": "string", "instructor": "string", "topics": ["string"], "price": number, "thumbnail_image": "string" null, "created_at": "string", "updated_at": "string" } null }</pre>	

GET /api/courses	
Request	
Header	Authorization: Bearer <token>

Query	q: string undefined // search based on title, topic, instructor page: number undefined // default = 1 limit: number undefined // default = 15, max = 50
Response	
<pre>{ "status": "success" "error", "message": "string", "data": [{ "id": "string", "title": "string", "instructor": "string", "description": "string", "topics": ["string"], "price": number, "thumbnail_image": "string" null, "total_modules": number, "created_at": "string", "updated_at": "string" }] null, "pagination": { "current_page": number, "total_pages": number, "total_items": number } }</pre>	

GET /api/courses/:id	
Request	
Header	Authorization: Bearer <token>
Response	
{	

```

    "status": "success" | "error",
    "message": "string",
    "data": {
      "id": "string",
      "title": "string",
      "description": "string",
      "instructor": "string",
      "topics": ["string"],
      "price": number,
      "thumbnail_image": "string" | null,
      "total_modules": number,
      "created_at": "string",
      "updated_at": "string"
    } | null
  }
}

```

PUT /api/courses/:id	
Request	
Header	Content-Type: multipart/form-data
	Authorization: Bearer <token>
Body	title: string description: string instructor: string topics: string[] price: number thumbnail_image: <binary image file> null
Response	

```
{
  "status": "success" | "error",
  "message": "string",
  "data": {
    "id": "string",
    "title": "string",
    "description": "string",
    "instructor": "string",
    "topics": ["string"],
    "price": number,
    "thumbnail_image": "string" | null,
    "created_at": "string",
    "updated_at": "string"
  } | null
}
```

DELETE /api/courses/:id

Request

Header

Authorization: Bearer <token>

Response

// empty (204 no content)

POST /api/courses/:id/buy

Request

Header

Authorization: Bearer <token>

Response

```
{
  "status": "success" | "error",
  "message": "string",
```

```

    "data": {
      "course_id": "string",
      "user_balance": number,
      "transaction_id": "string"
    } | null
  }
}

```

GET /api/courses/my-courses	
Request	
Header	Authorization: Bearer <token>
Query	q: string undefined // search based on title, topic, instructor page: number undefined // default = 1 limit: number undefined // default = 15, max = 50
Response	
<pre> { "status": "success" "error", "message": "string", "data": [{ "id": "string", "title": "string", "instructor": "string", "topics": ["string"], "thumbnail_image": "string" null, "progress_percentage": number, "purchased_at": "string" }] null, "pagination": { "current_page": number, "total_pages": number, "total_items": number } } </pre>	


```
}
```

2. CRUD Module

POST /api/courses/:courseId/modules	
Request	
Header	Content-Type: multipart/form-data
	Authorization: Bearer <token>
Body	title: string description: string pdf_content: <binary PDF file> null video_content:<binary Video file> null
Response	
<pre>{ "status": "success" "error", "message": "string", "data": { "id": "string", "course_id": "string", "title": "string", "description": "string", "order": number, "pdf_content": "string" null, "video_content": "string" null, "created_at": "string", "updated_at": "string" } null }</pre>	

GET /api/courses/:courseId/modules
Request

Header	Authorization: Bearer <token>
Query	page: number undefined // default = 1 limit: number undefined // default = 15, max = 50
Response	
<pre>{ "status": "success" "error", "message": "string", "data": [{ "id": "string", "course_id": "string", "title": "string", "description": "string", "order": number, "pdf_content": "string" null, "video_content": "string" null, "is_completed": "boolean", "created_at": "string", "updated_at": "string" }] null, "pagination": { "current_page": number, "total_pages": number, "total_items": number } }</pre>	

GET /api/modules/:id	
Request	
Header	Authorization: Bearer <token>
Response	

```
{
  "status": "success" | "error",
  "message": "string",
  "data": {
    "id": "string",
    "course_id": "string",
    "title": "string",
    "description": "string",
    "order": number,
    "pdf_content": "string" | null,
    "video_content": "string" | null,
    "is_completed": "boolean",
    "created_at": "string",
    "updated_at": "string"
  } | null
}
```

PUT /api/modules/:id	
Request	
Header	Content-Type: multipart/form-data
	Authorization: Bearer <token>
Body	title: string description: string pdf_content: <binary PDF file> null video_content:<binary Video file> null
Response	
<pre>{ "status": "success" "error", "message": "string", "data": { "id": "string", "course_id": "string", "title": "string", "description": "string",</pre>	

```

        "order": number,
        "pdf_content": "string" | null,
        "video_content": "string" | null,
        "created_at": "string",
        "updated_at": "string"
    } | null
}

```

DELETE /api/modules/:id

Request

Header	Authorization: Bearer <token>
---------------	-------------------------------

Response

// empty (204 no content)

PATCH /api/courses/:courseId/modules/reorder

Request

Header	Authorization: Bearer <token>
---------------	-------------------------------

Body	<pre> { "module_order": [{ "id": "string", "order": number }, { "id": "string", "order": number }, { "id": "string", "order": number } dst] } </pre>
-------------	--

Response

```

{
  "status": "success" | "error",
  "message": "string",

```

```

    "data": {
      "module_order": [
        { "id": "string", "order": number },
        { "id": "string", "order": number },
        { "id": "string", "order": number } dst
      ]
    } | null
  }
}

```

PATCH /api/modules/:id/complete

Request

Header	Authorization: Bearer <token>
---------------	-------------------------------

Response

```

{
  "status": "success" | "error",
  "message": "string",
  "data": {
    "module_id": "string",
    "is_completed": true,
    "course_progress": {
      "total_modules": number,
      "completed_modules": number,
      "percentage": number
    },
    "certificate_url": "string | null" // jika 100% complete
  } | null
}

```

3. Auth

POST /api/auth/login

Request

Header	Content-Type: application/json
Body	<pre>{ "identifier": "string", // email or username "password": "string" }</pre>
Response	
<pre>{ "status": "success" "error", "message": "string", "data": { "username": "string", "token": "string" } null }</pre>	

POST /api/auth/register	
Request	
Header	Content-Type: application/json
Body	<pre>{ "first_name": "string", "last_name": "string", "username": "string", "email": "string", "password": "string", "confirm_password": "string" }</pre>
Response	
<pre>{ "status": "success" "error", "message": "string", }</pre>	

```
"data": {
  "id": "string",
  "username": "string",
  "first_name": "string",
  "last_name": "string",
} | null
}
```

GET /api/auth/self

Request

Header

Authorization: Bearer <token>

Response

```
{
  "status": "success" | "error",
  "message": "string",
  "data": {
    "id": "string",
    "username": "string",
    "email": "string",
    "first_name": "string",
    "last_name": "string",
    "balance": number`
  } | null
}
```

4. CRUD User

GET /api/users

Request

Header	Authorization: Bearer <token>
Query	q: string // search based on username, firstname, lastname, email page: number undefined // default = 1 limit: number undefined // default = 15, max = 50
Response	
<pre>{ "status": "success" "error", "message": "string", "data": [{ "id": "string", "username": "string", "email": "string", "first_name": "string", "last_name": "string", "balance": number }] null, "pagination": { "current_page": number, "total_pages": number, "total_items": number } }</pre>	

GET /api/users/:id	
Request	
Header	Authorization: Bearer <token>
Response	
<pre>{ "status": "success" "error",</pre>	


```

    "message": "string",
    "data": {
      "id": "string",
      "username": "string",
      "email": "string",
      "first_name": "string",
      "last_name": "string",
      "balance": number,
      "courses_purchased": number
    } | null
  }
}

```

POST /api/users/:id/balance	
Request	
Header	Content-Type: application/json
	Authorization: Bearer <token>
Body	<pre> { "increment": number } </pre>
Response	
<pre> { "status": "success" "error", "message": "string", "data": { "id": "string", "username": "string", "balance": number } null } </pre>	

PUT /api/users/:id (pastikan admin tidak bisa diubah)

Request	
Header	Content-Type: application/json
	Authorization: Bearer <token>
Body	<pre>{ "email": "string", "username": "string", "first_name": "string", "last_name": "string", "password": "string" }</pre>
Response	
<pre>{ "status": "success" "error", "message": "string", "data": { "id": "string", "username": "string", "first_name": "string", "last_name": "string", "balance": number } null }</pre>	

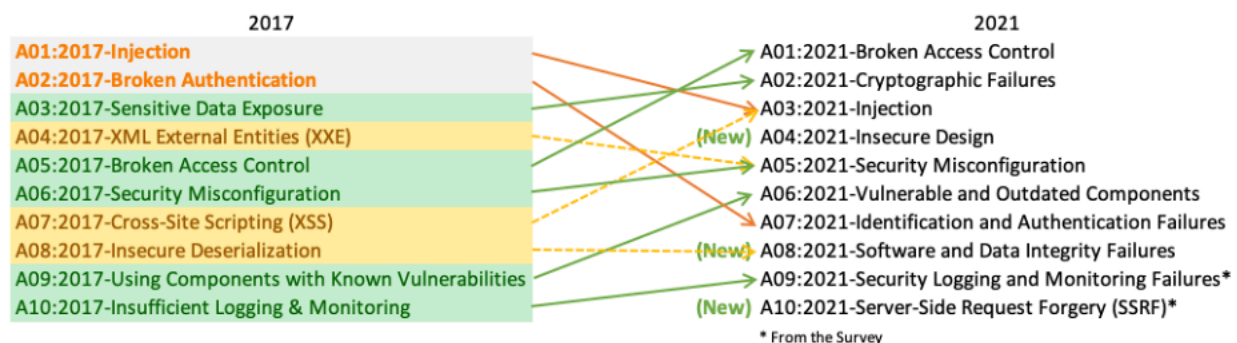
DELETE /api/users/:id (pastikan kalian tidak bisa menghapus admin)	
Request	
Header	Authorization: Bearer <token>
Response	
// 204 no content	

Bonus

Pada bagian ini, Anda dapat mengimplementasikan bonus - bonus yang disediakan **tanpa maksimal jumlah**. Gunakan bonus ini sebagai sarana untuk mengeksplorasi berbagai aspek dalam *web development* untuk memperdalam pengetahuan dan pemahaman Anda.

B01 - OWASP

OWASP adalah sebuah organisasi nirlaba yang fokus pada keamanan web app. OWASP banyak menyediakan sumber daya agar Anda bisa mempelajari lebih lanjut tentang keamanan web app. OWASP menyediakan daftar celah-celah keamanan yang rentan terjadi. Berikut adalah kategori OWASP top 10.



Tugas anda adalah memilih **3 dari 10** celah keamanan yang rentan terjadi di tahun 2021, kemudian buktikan bahwa website yang anda buat tidak dapat ditembus menggunakan celah keamanan tersebut. Pembuktian dapat dilakukan dengan melakukan simulasi serangan terhadap website anda dan **dokumentasikan dalam bentuk video**.

B02 - Deployment

Melakukan deployment untuk *monorepo* dan *database website* yang terhubung dengan baik. Ketentuan *deployment* dibebaskan sesuai kebutuhan peserta seleksi. Sertakan link deployment pada README.md.

B03 - Polling

Buatlah sebuah sistem *polling* sehingga setelah admin menambahkan *course*, tampilan pada user akan langsung *ter-update* tanpa melakukan *reload/refresh page*. *Polling* dapat dibuat dengan mengambil data setiap detik. Kalian bisa menggunakan *short-polling* atau *long-polling*, tetapi disarankan menggunakan *long-polling*.

B04 - Caching

Caching adalah teknik untuk menyimpan data sementara sehingga pemanggilan data yang sering diakses dapat dilakukan dengan lebih cepat. Dengan mengimplementasikan *caching*, beban pada database dapat dikurangi dan performa aplikasi dapat meningkat.

Redis adalah *in-memory* database yang cepat dan mendukung berbagai struktur data. Gunakan Redis untuk mengimplementasikan caching pada pemanggilan database di monolith bagian backend. Implementasikan caching untuk endpoint yang paling sering diakses dan sertakan cara pengujiannya dalam Readme. Pastikan pula bahwa kalian sudah melakukan invalidasi *cache* apabila terdapat perubahan pada data.

B05 - Lighthouse

Google Lighthouse adalah alat (*tools*) otomatis sumber terbuka yang digunakan untuk mengaudit atau mengukur kualitas suatu aplikasi website. Google Lighthouse dikembangkan oleh Google dan bertujuan untuk membantu pengembangan web (*developer*).

Google Lighthouse dapat dijalankan untuk menguji semua website, termasuk halaman web yang bersifat publik atau yang membutuhkan otentikasi. Google Lighthouse dapat menguji beberapa kategori pada suatu website, di antaranya yaitu performa, aksesibilitas, SEO, dan best practices. Jalankan Lighthouse untuk setiap tampilan halaman FE yang ada di monolith. Lakukan perbaikan pada kode hingga tiap-tiap halaman memiliki skor rata-rata **minimal 95**. Lampirkan tangkapan layar skor pada Readme.

B06 - Responsive Layout

Pada *monolith* bagian *frontend*, halaman dibuat *responsive* sesuai dengan ukuran layar gawai pengguna dengan menyesuaikan tampilan agar tetap terlihat tertata dan estetik. *Responsive* diterapkan pada tiap halaman di *monolith service* bagian *frontend*. (Keindahan UI tidak berpengaruh pada *layout* yang *responsive*)

B07 - Dokumentasi API

Developer yang baik adalah developer yang mendokumentasikan karya yang ia buat agar *readable* dan mudah dipahami oleh orang lain. Buatlah dokumentasi API untuk *single service* dan *monolith service* bagian *backend* menggunakan **swagger** dan sertakan link tersebut pada README.md.

B08 - SOLID

Karena kalian akan menggunakan OOP, terapkan prinsip SOLID pada Monolith BE ataupun REST API. Tuliskan pada README bagaimana cara kalian menerapkan prinsip tersebut secara jelas dan lengkap.

B09 - Automated Testing

Testing merupakan hal yang sering dilupakan oleh developer. Akan tetapi, hal ini merupakan hal yang sangat penting dalam pengembangan perangkat lunak. Automated testing banyak digunakan karena dapat mengurangi waktu testing dan memastikan spesifikasi perangkat lunak tetap terjaga.

Buatlah **unit test** atau **automated end-to-end testing** pada salah satu FOX:

- Untuk BE berarti *unit test*
- Untuk FE berarti *end-to-end test*

Test seminimalnya dilakukan pada **60%** bagian kode:

- End-to-end test: 60% jumlah page (4 dari 6 halaman) dengan mencakup seluruh fungsionalitas dari halaman tersebut
- Unit test: perlu menghasilkan 60% coverage pada seluruh file/kelas pada layer services/business/use case (pastikan untuk menaruh logic sistem pada layer tersebut)

B10 - Fitur Tambahan

Berikut adalah ide-ide yang diberikan sebagai referensi untuk bonus fitur tambahan. Jika memang memiliki ide fitur yang lainnya, silakan berkreasi untuk diimplementasikan. Sebagai catatan, jangan lupa untuk menambahkan bonus fitur tambahan yang kalian kerjakan dalam README.md repository terkait. Khusus bagian bonus fitur tambahan ini, **maksimal jumlah bonus fitur tambahan yang diimplementasikan adalah 3**. Nilai yang didapatkan dari bagian ini sangat bergantung terhadap kompleksitas fitur yang diimplementasikan.

Berikut merupakan beberapa ide bonus fitur tambahan yang dapat ditambahkan:

1. OAuth

Tambahkan autentikasi menggunakan OAuth (minimal melalui Google) sebagai alternatif login dan registrasi, tanpa menggantikan sistem autentikasi utama yang sudah ada.

2. Quiz System

Tambahkan sistem quiz pada setiap akhir module atau course:

- Admin dapat membuat quiz dengan multiple choice (implementasikan melalui *interface* sendiri)
- User harus mendapatkan nilai minimal tertentu untuk lanjut ke module berikutnya
- Nilai quiz tercatat dan mempengaruhi sertifikat

3. Shopping Cart

Implementasikan fitur keranjang belanja:

- User dapat menambahkan beberapa course ke keranjang
- Checkout sekaligus dengan total pembayaran
- Riwayat transaksi yang detail

4. Course Rating & Review

Pada Course Detail Page, terdapat fitur untuk:

- Memberikan rating (bintang 1-5)
- Menulis review tentang course
- Melihat rating dan review dari user lain
- Filter course berdasarkan rating

5. Discussion Forum

Tambahkan forum diskusi pada setiap course:

- User dapat bertanya tentang materi
- User lain atau instructor dapat menjawab
- Sistem upvote/downvote untuk jawaban

6. Progress Tracking Dashboard

Halaman dashboard yang menampilkan:

- Grafik progress pembelajaran
- Statistik waktu belajar
- Achievement/badges system
- Learning streak counter

B11 - Bucket

Bucket adalah cloud object storage service yang menyediakan tempat untuk menyimpan objek-objek seperti images, videos, binaries, dan file-file lainnya. Dengan menggunakan bucket, kalian akan menyimpan film yang diunggah oleh pengguna di bucket pilihan kalian sehingga penyimpanan source code dan media course terpisah pada server.

Kalian dapat menggunakan cloud provider pilihan kalian seperti AWS Simple Storage Service (S3), Google Cloud Bucket, Cloudflare R2, Azure Blob Storage, atau lainnya. Implementasi **tidak diperbolehkan** untuk menambah atau mengubah body dari API *course*.

Batasan dan Deliverables

Batasan

Berikut adalah batasan-batasan yang **wajib** dipenuhi oleh website yang akan dibuat:

1. Menggunakan bahasa **PHP, Python, Typescript, Java, Go**.
2. Diperkenankan menggunakan framework, framework yang dapat digunakan adalah **Laravel, Django, NestJS, Spring Boot, dan Gin**.
3. Meskipun menggunakan framework, untuk Frontend harus diimplementasi dengan menggunakan **vanilla HTML + template engine**.
4. Menggunakan **relational database**, seperti MySQL, PostgreSQL, MariaDB. Diwajibkan untuk menggunakan **ORM** (Object Relational Mapper) dan membuat automasi **seeding dan migration** agar mempermudah aplikasi ketika dijalankan.
5. Wajib menggunakan konsep **Object Oriented Programming (OOP)**. Silahkan cari tutorial yang ada di internet untuk penggunaannya pada masing-masing bahasa.
6. Wajib mengimplementasi **minimal 3 design pattern**. Tuliskan design pattern yang digunakan pada README.md beserta alasan mengapa menggunakan design pattern tersebut.
7. Monolith wajib untuk di-**dockerize**. Buat file **docker-compose.yml** agar bisa langsung dijalankan beserta dengan databasenya. Anda bisa mengikuti tutorial yang ada di internet.
8. Pengerjaan dilakukan secara **individu**. Segala jenis plagiarisme terhadap sesama peserta seleksi dapat menyebabkan gugurnya status kalian sebagai peserta.

Deliverables

- Batas akhir pengerjaan adalah **Jumat, 22 Agustus 2025 pada jam 22.10 WIB**
- Pengumpulan dilakukan melalui link [berikut](#).
- Peserta seleksi memiliki hasil akhir **satu GitHub repository**. Repository di-public setelah pengumpulan, maksimum sehari setelah deadline.
- Segala bentuk pertanyaan dapat ditanyakan melalui link [QnA](#)
- Wajib membuat **README.md** pada repository. Penjelasan mencakup:
 - Identitas diri yaitu **Nama dan NIM**
 - Cara menjalankan aplikasi
 - Design pattern yang digunakan dan alasannya
 - Technology stack beserta versi yang dipakai
 - Endpoint apa saja yang dibuat
 - Bonus yang dikerjakan bila ada
 - Screenshot aplikasi

- Form pengumpulan akan diberikan di kemudian hari